

Unfold

Technical report - opportunity discovery for the ALU community

Rajveer Singh Jolly

African Leadership University - Mobile Application Development

Unfold: An Opportunity Platform for the ALU Community

Mobile Application Development - Technical Report

Student: Rajveer Singh Jolly

Platform: Flutter for Android

Backend: Firebase

Repository: <https://github.com/SpaceMan619/unfold-alu>

Abstract

Unfold is a role-aware mobile platform that connects African Leadership University students with student-led ventures. It replaces fragmented opportunity posts and informal application tracking with one workflow for discovery, evidence-based matching, applications, publishing, and applicant review. Flutter provides the mobile interface, Riverpod coordinates state, Firebase Authentication manages identity, and Cloud Firestore stores persistent data and provides live updates. Firebase AI Logic sends a user-selected PDF directly to Gemini and stores only editable structured results rather than the source file. The interface uses a neutral glass-inspired visual system designed for Android performance. This report explains the product reasoning, architecture, data model, security rules, testing, optimization decisions, limitations, and future development.

1. Problem, Users, and Product Reasoning

ALU students often find practical roles through disconnected channels such as group chats, email, and personal networks. Student founders need help in software, design, marketing, research, operations, and community work, but may not have a structured recruitment process. ALU's mission-driven education makes this more than a generic job-board problem: the platform should help students build evidence of experience while helping early ventures find relevant contributors [1].

Unfold serves two users:

1. Students discover roles, save them, analyze a CV, inspect explainable matches, apply, and track progress.
2. Founders publish paid, unpaid, or volunteer roles and manage applicants through a visible pipeline.

The main differentiator is CV-assisted but human-controlled matching. AI extracts evidence from a PDF into skills, suggested roles, and a short summary. A deterministic score then compares editable skills with each role's required skills. AI does not accept or reject candidates.

2. Implemented Workflows

2.1 Authentication and onboarding

Firebase Authentication supports Google and email/password sessions. Google onboarding is restricted in application logic to ALU student or education domains. `SessionController` observes authentication state, restores the corresponding Firestore profile, and exposes explicit loading, signed-out, role-selection, onboarding, and authenticated stages. `AuthGate` renders the correct screen from that state instead of manually coordinating a stack of authentication routes.

2.2 Student experience

The Discover feed supports search, category filters, compensation filters, pull-to-refresh, saving, and opportunity details. A student can open the validated application form, provide motivation and availability, add an optional portfolio link, and submit. Submission creates an application only after validation; opening the form never creates one. Applications are shown with one of six states: submitted, reviewing, shortlisted, waitlisted, accepted, or rejected.

The profile is an external-view preview. The student can edit bio, class year, links, skills, and interests while name and email remain identity fields. Profile images are restricted, cropped, and compressed before being stored. The CV workflow accepts one PDF of at most 8 MB, displays analysis progress, allows a replacement CV, and lets the student remove extracted skills.

2.3 Founder experience

Founder mode provides opportunity publishing and an applicant pipeline. A listing records title, venture, description, required skills, location, commitment, category, compensation type, optional monthly amount and currency, contact email, and optional deadline. Paid listings require a positive amount in RWF or USD. The founder can change applicant status and open a pre-addressed email. Writes are linked to Firestore through owner and founder IDs rather than being temporary UI state.

3. Architecture and State Management

Unfold uses a feature-first structure with four layers:

```
Flutter view and reusable widgets
-> Riverpod provider or Notifier controller
-> repository interface
-> Firebase implementation
```

`main.dart` initializes Firebase and places the app inside `ProviderScope`. Views use `ref.watch` for values that should rebuild the UI and `ref.read(...notifier)` for commands. Controllers own workflow state, validation transitions, optimistic changes, and rollback. Repository interfaces own persistence contracts. Firestore syntax therefore remains outside most widgets, and tests can replace a live repository with an in-memory fake.

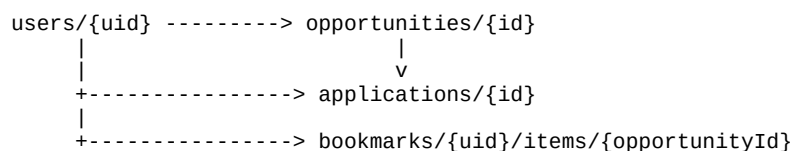
Important controllers include:

- `SessionController` for identity, role selection, profile restoration, and sign-out.
- `OpportunityActivityController` for active opportunities, saved IDs, applied IDs, submission, and refresh.
- `ApplicationReviewController` for founder-side applicant streams and status updates.
- `CvAnalysisController` for PDF selection, AI stages, structured extraction, editing, and Firestore persistence.

This is preferable to global `setState` because authentication, discovery, applications, and profile evidence outlive individual widgets. Small presentational values such as a search query remain local state.

4. Firebase Backend and Schema

Firebase Authentication identifies the current user. Cloud Firestore snapshot listeners keep opportunities, applications, bookmarks, and profiles synchronized [3], [4]. The active schema is deliberately small.



4.1 users/{uid}

Stores name, email, role, profile fields, editable skills and interests, compressed photo data, `cvFileName`, `cvSummary`, `cvSuggestedRoles`, `cvAnalyzedAt`, and timestamps.

4.2 opportunities/{opportunityId}

Stores `ownerId`, role and venture details, skills, category, location, commitment, `isPaid`, `monthlyAmount`, currency, contact email, status, optional deadline, and `createdAt`.

deadline has one source of truth: an optional Firestore timestamp. The client parses the timestamp and calculates presentation labels such as days remaining. Local fallback opportunities do not invent deadlines, so a closing label appears only when persisted backend data contains one.

4.3 applications/{applicationId}

Stores `opportunityId`, `studentId`, `founderId`, motivation, availability, portfolio URL, status, student summary fields, `isDemo`, and `createdAt`. Both participant IDs make student and founder queries direct.

4.4 bookmarks/{uid}/items/{opportunityId}

Stores a private saved-opportunity reference below its owner.

5. Security and Data Integrity

Security is enforced in Firestore rules rather than by hidden buttons [5]. A user creates only their own profile with an authenticated ALU-domain email. Application creation requires the signed-in student ID and initial submitted status. For a live opportunity, its owner must match the submitted founder ID. Only the owning founder or an administrator can change status, and relationship fields remain immutable. Bookmark access is limited to the matching UID.

Opportunity creation requires authenticated ownership. Updates and deletes require the same owner Id; this supports the app's accessible founder-mode toggle without relying on a mutable client role as the security boundary. Rules also require a deadline, when present, to be a Firestore timestamp.

No service-account credential, provider secret, App Check debug token, or signing key belongs in the repository. Firebase client configuration identifies the public Android client and is not an authorization secret. Debug builds use App Check's debug provider and release builds use Play Integrity.

6. CV Intelligence and Explainable Matching

CvAnalysisController uses `file_picker` to select a PDF, rejects unreadable data and files above 8 MB, and sends the bytes as inline PDF data through Firebase AI Logic to the configured Gemini model. A response schema requires three fields: up to eight skills, up to three suggested roles, and a one-sentence summary. The system instruction says to extract only evidence present in the document and not infer sensitive traits.

Only structured results and the filename are merged into the authenticated user document. The raw PDF is not uploaded to Cloud Storage, which reduces retained personal data and avoids depending on a paid Storage bucket. Students can remove an extracted skill or replace the CV.

Matching is deterministic and explainable:

```
score = matched required skills / total required skills
```

Normalization makes comparison case-insensitive and trims spacing. The detail sheet shows the evidence behind the result. The score is a discovery aid, not a hiring prediction. If no profile evidence exists, Unfold asks the student to add skills or analyze a CV instead of displaying a fabricated percentage.

7. UI/UX Decisions and Performance

The interface uses a neutral black-and-white base, venture-specific accent colors, rounded editorial cards, large touch targets, and a floating pill navigation bar. The profile resembles an external public profile rather than a settings form. Opportunity cards prioritize the first-glance facts requested by students: role, venture, category, location, commitment, compensation, deadline when real, and match evidence.

Early builds applied live backdrop blur and restarted a match-ring animation whenever list items rebuilt. Physical-device testing exposed jitter during scrolling. Version 1.0 therefore makes three targeted changes:

- MatchRing is static and paints the already calculated value without a 900 ms rebuild animation.
- Discovery caches normalized search text and match results. The cache refreshes only when opportunity data or profile skills change, not while scrolling.
- The rotating headline owns its timer in a small child widget, and the navigation surface uses static translucent paint rather than a live backdrop filter.

These changes preserve depth and visual identity while reducing repeated CPU, layout, and GPU work. Pull-to-refresh remains the explicit user action for fetching current provider data.

8. CRUD, Dynamic Updates, and Failure Handling

Founder publishing demonstrates create; discovery streams demonstrate read; opportunity editing and applicant status transitions demonstrate update; founder removal and student withdrawal paths demonstrate delete. Firestore snapshots propagate persistent changes back into Riverpod state and the interface.

Optimistic updates keep interactions immediate. For example, an application status changes locally before the repository call completes. If Firestore rejects the write, the controller restores the previous list and surfaces an error. Forms validate required values and compensation rules. AI analysis presents readable errors for quota, App Check, unsupported location, malformed output, and network failure.

When Firestore contains no active opportunities, sourced ALU-relevant fallback content keeps the discovery experience understandable. These entries are clearly demonstration content. They do not carry fabricated deadlines, historical match scores, or application records.

9. Testing and Verification

The test suite covers domain mapping, onboarding, session restoration, search/filter behavior, opportunity creation, deadline rendering, application validation and submission, optimistic founder review, explainable matching, and important widget flows. Riverpod provider overrides replace Firebase dependencies with controlled fakes.

Release verification uses:

```
dart format --output=none --set-exit-if-changed lib test
flutter analyze
flutter test
flutter build apk --release --target-platform android-arm64
```

Static analysis checks the configured lints, tests exercise state transitions, and a physical Android device is used for final interaction and performance testing. Firestore rules are compiled and deployed with the Firebase CLI. Emulator-based rule tests and a wider device matrix remain future hardening work.

10. Maintainability and Scalability

Feature folders keep models, controllers, repositories, views, and sheets close to their domain. Reusable widgets such as GlassSurface and MatchRing prevent repeated styling and logic. Short comments explain only non-obvious boundaries, including cache invalidation and the deterministic match formula.

At larger scale, the current full active-opportunity stream should become paginated indexed queries. Search would move to a dedicated service, notifications to Cloud Functions and Cloud Messaging, and administrator authority to trusted custom claims. App Check enforcement, analytics, accessibility audits, rule-emulator tests, and media hosting would be production requirements. The repository boundary permits these changes without rewriting the view layer.

11. Challenges and Lessons Learned

The first challenge was balancing a distinctive visual system with smooth Android performance. Device evidence showed that more blur and animation did not mean a better interface; profiling led to a lighter implementation. The second was preserving truth across a fallback feed and live Firebase data. Removing generated fallback deadlines made the UI simpler and more trustworthy. The third was fitting CV intelligence into a no-Storage architecture. Sending a selected PDF directly through Firebase AI Logic and retaining only structured evidence reduced storage cost and privacy exposure.

The main engineering lesson was separation of concerns. Riverpod controllers make state transitions visible, repositories isolate backend details, and security rules remain the final authority. Another lesson was that an AI feature is stronger when its limits are explicit and the user can edit its output.

12. Limitations and Future Improvements

Version 1.0 has the following boundaries:

1. Fallback opportunity content is presentation data, not a claim of a live vacancy.
2. Email actions open the device mail client; Unfold does not provide in-app messaging.
3. There is no administrator mobile interface for institution-level venture verification.
4. Production needs App Check enforcement, rule-emulator tests, pagination, accessibility review, and broader device testing.
5. AI output can be incomplete or wrong, so extracted evidence remains editable and matching remains advisory.

Future work should prioritize institution-managed founder verification, notifications, server-side search, privacy controls for CV-derived data, and analytics that measure whether applications receive responses.

13. Conclusion

Unfold converts a fragmented campus process into a coherent student-to-founder workflow. Its originality comes from combining ALU-specific identity, founder publishing, structured applicant management, editable CV intelligence, and explainable matching in a purpose-built mobile experience. Its engineering foundation uses Firebase persistence and authorization, Riverpod state separation, repository boundaries, validated workflows, and device-driven performance optimization. Version 1.0 is a functional foundation that makes both its live capabilities and its remaining limitations clear.

References

[1] African Leadership University, "About ALU," ALU. [Online]. Available: <https://www.alueducation.com/about/>. [Accessed: 13-Jul-2026].

[2] Flutter, "Flutter architectural overview," Flutter Documentation. [Online]. Available: <https://docs.flutter.dev/resources/architectural-overview>. [Accessed: 13-Jul-2026].

[3] Firebase, "Manage users in Firebase," Firebase Authentication Documentation. [Online]. Available: <https://firebase.google.com/docs/auth/flutter/manage-users>. [Accessed: 13-Jul-2026].

[4] Firebase, "Get realtime updates with Cloud Firestore," Firebase Documentation. [Online]. Available: <https://firebase.google.com/docs/firestore/query-data/listen>. [Accessed: 13-Jul-2026].

[5] Firebase, "Get started with Cloud Firestore Security Rules," Firebase Documentation. [Online]. Available: <https://firebase.google.com/docs/firestore/security/get-started>. [Accessed: 13-Jul-2026].

[6] Firebase, "Firebase AI Logic," Firebase Documentation. [Online]. Available: <https://firebase.google.com/docs/ai-logic>. [Accessed: 13-Jul-2026].

[7] Riverpod, "Providers," Riverpod Documentation. [Online]. Available: <https://riverpod.dev/docs/concepts2/providers>. [Accessed: 13-Jul-2026].

[8] Firebase, "Connect your app to the Cloud Firestore Emulator," Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/emulator-suite/connect_firestore. [Accessed: 13-Jul-2026].

Application screenshots

Final release-mode discovery feed and opportunity detail workflow.

